

Mongoose crude

`populate()` ki zarurat kyu padi.

1. Sabse pehle: Relationship kya hai?

Maan lo hamare application me **User** aur **Post** hain.

Ek user bahut saare posts bana sakta hai.

User



Post



Post



Post

Example:

Mongoose crude

!-d097aeba1769

User collection

_id	name
U101	Tarun
U102	Rahul

Post collection

_id	title	user
P1	JavaScript	U101
P2	MongoDB	U101
P3	React	U102

Yahan `Post.user` me **User ki ID** rakhi hai.

2. Post me User ki ID hi kyu rakhi?

Hum Post ke andar pura user object store kar sakte the:

`</>` JavaScript

```
{
  title: "MongoDB",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com",
    age: 22
  }
}
```

Lekin ye achha design nahi hai.

Kyun?

Maan lo Tarun ke **1000 posts** hain.

Maan lo Tarun ke **1000 posts** hain.

To 1000 documents me:

```
name: Tarun
email: ...
age: ...
```

repeat hoga.

Agar Tarun apna email change kare:

```
tarun@gmail.com
    ↓
new@gmail.com
```

to potentially bahut saare documents update karne padenge.

Isliye hum sirf **User ki ID** store karte hain:

```
</> JavaScript

{
  title: "MongoDB",
  user: "U101"
}
```

Aur User ki actual information User collection me rahegi.

3. Ab problem kya aayi?

Ab hum Post nikalte hain:

```
⟨/⟩ JavaScript
```

```
const posts = await Post.find();
```

Result:

```
⟨/⟩ JavaScript
```

```
[
  {
    title: "JavaScript",
    user: "U101"
  },
  {
    title: "MongoDB",
    user: "U101"
  }
]
```

Hume pata hai:

Hume pata hai:

```
U101 = Tarun
```

Lekin response me sirf:

```
user: U101
```

aa raha hai.

Frontend ko chahiye:

```
</> JavaScript  
  
{  
  title: "MongoDB",  
  user: {  
    name: "Tarun",  
    email: "tarun@gmail.com"  
  }  
}
```

To ab question hai:

U101 ki actual details kaise laayenge?

Yahin `populate()` kaam aata hai.

4. Populate kya karta hai?

Simple definition:

`populate()` kisi document me stored reference ID ko use karke related document ki actual information fetch karta hai.

Example:

`</>` JavaScript



```
const posts = await Post.find().populate("user");
```

Mongoose dekhega:

```
</> JavaScript
```

```
user: "U101"
```

Aur schema me dekhega:

```
</> JavaScript
```

```
ref: "User"
```

To Mongoose samjhega:

U101

↓

User collection

↓

U101 wala document

Aur result me user ki details laga dega.



5. Without populate vs With populate

Without `populate()`

</> JavaScript

```
const posts = await Post.find();
```

Result:

</> JavaScript

```
{  
  title: "MongoDB",  
  user: "U101"  
}
```

Sirf ID mili.

With `populate()`

```
</> JavaScript

const posts = await Post.find().populate("user");
```

Result:

```
</> JavaScript

{
  title: "MongoDB",
  user: {
    _id: "U101",
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

Ab ID ki jagah related User document mil gaya.

6. Populate actually kaise pata karta hai ki U101 User hai?

Ye bahut important hai.

Schema me hum likhte hain:

```
</> JavaScript

const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});
```



Yahan 3 important cheezein hain:

type

```
</> JavaScript
```

```
type: mongoose.Schema.Types.ObjectId
```

Matlab:

`user` field me kisi document ki ID store hogi.

ref

```
</> JavaScript
```

```
ref: "User"
```

Matlab:

Ye ID **User model** ke document ko refer karti hai.

Ab jab:

```
</> JavaScript  
  
.populate("user")
```

likha:

Mongoose samajh gaya:

```
Post ka "user" field  
    ↓  
isme User ki ObjectId hai  
    ↓  
User model me jao  
    ↓  
matching document lao
```

7. Real-life example

Maan lo tumhare paas ek **college database** hai.

Student

```
</> JavaScript  
  
{  
  _id: 101,  
  name: "Tarun"  
}
```

Course

```
</> JavaScript  
  
{  
  _id: 501,  
  name: "MongoDB",  
  student: 101  
}
```



Course me humne:

```
student = 101
```

store kiya.

Lekin agar API response me chahiye:

JavaScript

```
{  
  name: "MongoDB",  
  student: {  
    name: "Tarun"  
  }  
}
```

to:

to:

```
</> JavaScript
```

```
Course.find().populate("student")
```

use kar sakte hain.

8. SQL JOIN se relation

Agar tum SQL jaante ho, to aur easy hai.

SQL me:

```
</> SQL
```

```
SELECT *  
FROM posts  
JOIN users  
ON posts.user_id = users.id;
```

MongoDB + Mongoose me roughly:

8. SQL JOIN se relation

Agar tum SQL jaante ho, to aur easy hai.

SQL me:

`</> SQL`

```
SELECT *  
FROM posts  
JOIN users  
ON posts.user_id = users.id;
```

MongoDB + Mongoose me roughly:

`</> JavaScript`

```
Post.find().populate("user");
```

Dono ka basic purpose:

Related data ko saath me lana

Hai.

Lekin technically `populate()` SQL JOIN exactly nahi hai. Mongoose related documents fetch karke references ko replace karta hai.

9. Populate ki zarurat kyu padi?

Main reason hai data ko duplicate na karna + relationship maintain karna.

Suppose Tarun ke 100 posts hain.

Bad approach

Har Post me:

 JavaScript 

```
{  
  title: "Post 1",  
  user: {  
    name: "Tarun",  
    email: "tarun@gmail.com"  
  }  
}
```



```
{
  title: "Post 2",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

JavaScript

```
{
  title: "Post 3",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

User information repeat ho rahi hai.

Better approach

Post me:

JavaScript

```
{
  title: "Post 1",
  user: ObjectId("...")
}
```

JavaScript

```
{
  title: "Post 2",
  user: ObjectId("...")
}
```

JavaScript

```
{
  title: "Post 3",
  user: ObjectId("...")
}
```

Aur User ki information:

```
</> JavaScript

{
  _id: ObjectId("..."),
  name: "Tarun",
  email: "tarun@gmail.com"
}
```

Ek hi jagah stored hai.

Jab actual user details chahiye:

```
</> JavaScript

.populate("user")
```

10. Ek complete example

User Model

</> JavaScript

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});

const User = mongoose.model("User", userSchema);
```

Post Model

</> JavaScript

```
const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
}
```



Mongoose crude

</> JavaScript

```
const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});

const Post = mongoose.model("Post", postSchema);
```

Data

User:

</> JavaScript

```
{
  _id: ObjectId("111"),
  name: "Tarun",
  email: "tarun@gmail.com"
}
```



Post:

JavaScript

```
{
  _id: ObjectId("222"),
  title: "MongoDB Populate",
  user: ObjectId("111")
}
```

Query

JavaScript

```
const post = await Post
  .find()
  .populate("user");
```

Result

JavaScript

```
{
  _id: "222",
  title: "MongoDB Populate",

  user: {
    _id: "111",
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

11. Populate me "user" hi kyu likha?

Schema:

```
</> JavaScript

user: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
}
```

Query:

```
</> JavaScript

.populate("user")
```

Yahan "user" Post ke field ka naam hai.

Agar schema hota:

```
</> JavaScript
```

```
author: {  
  type: mongoose.Schema.Types.ObjectId,  
  ref: "User"  
}
```

to:

```
</> JavaScript
```

```
.populate("author")
```

likhte.

So:

```
Schema field  
  ↓  
populate("same field")
```

12. Specific fields bhi la sakte ho

Maan lo User me:

```
</> JavaScript

{
  name: "Tarun",
  email: "tarun@gmail.com",
  password: "123456"
}
```

Password nahi chahiye.

To:

```
</> JavaScript

Post.find().populate("user", "name email");
```

Result:

Result:

```
</> JavaScript

{
  title: "MongoDB",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

Ye security ke liye bhi important hai.

